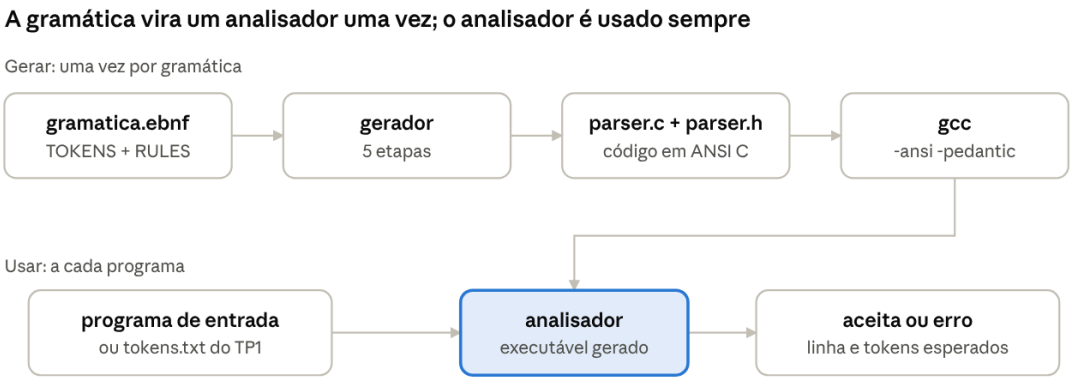
Manual do TP2: Gerador de Analisadores Sintáticos

Oct 21, 2026 · @Amaury Antônio de Castro Junior

1. Visão geral

O gerador lê uma gramática escrita em EBNF e produz o código-fonte, em ANSI C, de um analisador sintático descendente recursivo para a linguagem que ela descreve. O trabalho tem dois momentos: gerar o analisador, uma vez por gramática, e usá-lo para analisar programas.



fluxo do trabalho · gerar e usar

O gerador nunca lê o programa do usuário: ele só escreve o código do analisador. Quem lê o programa, token a token, é o analisador gerado.

Este manual mostra como compilar e executar o esqueleto, escrever gramáticas e testes e implementar, em ordem, as partes marcadas com `TODO(aluno)`. Os requisitos e a avaliação estão no enunciado do trabalho. Se houver divergência, vale o enunciado.

2. Conteúdo do pacote

O arquivo `tp2-gerador-ebnf.zip` contém o esqueleto completo. Ele compila e executa de ponta a ponta, mas o analisador gerado ainda não reconhece nenhuma entrada: completar isso é o trabalho.

Caminho	O que contém	Você altera?
`src/main.c`	Ordem das cinco etapas e opções de linha de comando	Não
`src/util.c`, `src/bitset.c`, `src/gramatica.c`	Alocação, mensagens de erro, conjuntos de bits e a gramática em memória	Não
`src/ebnf_lexer.c`	Léxico da notação EBNF	Não

Caminho	O que contém	Você altera?
`src/ebnf_parser.c`	Leitura das seções e das regras	Sim: as expressões
`src/verificacao.c`	Verificações semânticas	Sim: três verificações
`src/conjuntos.c`	Anulável, FIRST e FOLLOW	Sim: os três cálculos
`src/ll1.c`	Verificação LL(1)	Sim: as condições
`src/codegen.c`	Geração do `.c` e do `.h`	Sim: tradução, expressões regulares e `--tokens`
`src/microc_mapa.c`	Correspondência entre a gramática e o `tokens.h`	Sim: leitura do `tokens.h`
`microc/`	Adaptador do scanner do TP1 (`scanner.c`, `scanner.h`)	Sim: ajustes ao seu TP1
`testes/`	Gramáticas, programas, saídas esperadas e o script de testes	Sim: acrescente casos
`Makefile`	Compilação, testes e limpeza	Só a variável `LDLIBS_MICROC`, se precisar
`README.md`, `USO_IA.md`	Identificação do grupo e registro de uso de IA	Sim

Os pontos a implementar estão marcados no código com o comentário `TODO(aluno)`, que traz dicas sobre o que fazer. Para listá-los, use `grep -rn "TODO(aluno)" src microc`.

Comece lendo `src/gramatica.h`. Ele descreve a árvore usada para representar cada regra, que todas as etapas leem e completam.

3. Preparando o ambiente

O trabalho é desenvolvido e corrigido em Linux. No Windows, use o WSL (Ubuntu); no macOS, instale as ferramentas pelo Homebrew. São necessárias quatro ferramentas:

Ferramenta	Para que serve	Instalação no Ubuntu/WSL
`gcc`	Compilar o gerador e os analisadores gerados	`sudo apt install build-essential`
`make`	Automatizar a compilação e os testes	vem com `build-essential`
`flex`	Compilar o scanner do TP1 (modo Micro C)	`sudo apt install flex`
`valgrind`	Procurar vazamentos e acessos inválidos à memória	`sudo apt install valgrind`

Para conferir a instalação, rode `gcc --version`, `make --version` e `flex --version`. Os três devem responder com um número de versão.

Descompacte o pacote em uma pasta sem espaços ou acentos no caminho e entre nela:

```
unzip tp2-gerador-ebnf.zip
cd tp2-gerador-ebnf
```

Todos os comandos deste manual são executados a partir dessa pasta, a raiz do projeto.

4. Compilando e executando o gerador

O comando `make` compila o gerador e cria o executável `./gerador` na raiz do projeto. A compilação usa `gcc -ansi -pedantic -Wall -Wextra`, e o trabalho exige zero avisos. Trate cada aviso como erro.

Comando	O que faz
`make`	Compila o gerador e, se o TP1 estiver em `microc/`, também o analisador de Micro C
`make test`	Executa todos os testes automáticos
`make microc`	Gera e compila o analisador de Micro C
`make clean`	Apaga a pasta `build/` e o executável

O gerador recebe o arquivo da gramática e um prefixo de saída. Com `-o build/expr`, ele grava `build/expr.c` e `build/expr.h`:

```
./gerador gramatica.ebnf -o prefixo [--scanner microc] [--sets] [--tree] [--debug]
```

Opção	Efeito
`-o prefixo`	Gera `prefixo.c` e `prefixo.h`
`--sets`	Imprime a tabela de anulável, FIRST e FOLLOW
`--debug`	Imprime a gramática como foi lida, útil para conferir a leitura
`--tree`	Inclui no analisador gerado o suporte à impressão da árvore de derivação
`--scanner microc`	Usa o scanner do TP1 em vez de gerar um léxico (seção 7)

O que o esqueleto faz hoje

Logo depois de descompactar, o gerador já roda, mas avisa onde ainda falta código. Com a gramática de expressões, a saída é esta:

```
$ ./gerador testes/gramaticas/expr.ebnf -o build/expr --sets
testes/gramaticas/expr.ebnf:8:12: aviso: parse_expr ainda nao implementado: regras
lidas como vazias
NAO TERMINAL  ANULAVEL  FIRST / FOLLOW
expr          nao       FIRST  = {}
                        FOLLOW = {}
```

```
term          nao       FIRST  = {}
                        FOLLOW = {}
factor        nao       FIRST  = {}
                        FOLLOW = {}
```

O aviso aponta a primeira parte a implementar, e os conjuntos vazios mostram que o cálculo ainda não existe. Conforme você completa cada etapa, os avisos somem e os conjuntos são preenchidos. A seção 9 mostra a saída esperada.

5. Escrevendo uma gramática EBNF

Um arquivo `.ebnf` tem duas seções. Em `TOKENS` ficam os terminais, definidos por expressões regulares entre barras, e as diretivas `SKIP` para o que deve ser ignorado. Em `RULES` ficam as regras, e a primeira delas define o símbolo inicial. Este é o exemplo do pacote, `testes/gramaticas/expr.ebnf`:

```
(* Expressoes aritmeticas *)
TOKENS
  NUM  = /[0-9]+/ ;
  ID   = /[a-zA-Z_][a-zA-Z0-9_]*/ ;
  SKIP = /[ \t\r\n]+/ ;

RULES
  expr   = term { ( "+" | "-" ) term } ;
  term   = factor { ( "*" | "/" ) factor } ;
  factor = NUM | ID | "(" expr ")" ;
```

Três convenções evitam a maioria dos erros:

- Terminais com nome usam só maiúsculas (`NUM`, `ID`). Não terminais começam com minúscula (`expr`).
- Símbolos e palavras fixas da linguagem são escritos entre aspas (`"while"`, `"+"`) e não precisam ser declarados em `TOKENS`.
- Toda regra termina com ponto e vírgula.

Erros comuns e o que o gerador responde

As mensagens abaixo são as da solução completa. Cada uma traz arquivo, linha e coluna.

Erro na gramática	Mensagem
Esquecer o `;` no fim da regra anterior	`a.ebnf:5:7: erro: esperado ';', encontrado '='`
Usar um não terminal que não tem regra	`b.ebnf:4:21: erro: nao terminal item usado mas nao definido`
Usar um terminal com nome que não está em `TOKENS`	`c.ebnf:4:21: erro: terminal ID nao declarado em TOKENS`
Misturar maiúsculas e minúsculas no nome (`Lista`)	`erro: identificador 'Lista' invalido: terminais usam so maiusculas e nao terminais comecam com minuscula`

Erro na gramática	Mensagem
Escrever uma regra que nunca é usada	`f.ebnf:6:3: aviso: nao terminal sobra inalcancavel a partir de lista`
Repetição que pode não consumir nada: `{ [ "," ] }`	`erro: conflito LL(1) em lista: o conteudo de { } e anulavel`

No primeiro caso, a mensagem aponta a linha seguinte à do erro: sem o `;`, o gerador leu o nome da regra seguinte como parte da anterior e só estranhou ao encontrar o `=`. Quando um erro parecer sem sentido, olhe também a linha anterior.

Avisos não impedem a geração. Erros impedem, e nenhum arquivo é escrito.

6. Gerando, compilando e executando o analisador

O caminho completo tem três comandos: gerar o código, compilá-lo e executar o analisador sobre um programa. A pasta de saída precisa existir antes. Ela é criada pelo `make` ou com `mkdir -p build`.

```
./gerador testes/gramaticas/expr.ebnf -o build/expr --tree
gcc -ansi -pedantic -Wall -Wextra -o build/expr_bin build/expr.c
./build/expr_bin exemplo.txt
```

O analisador lê o programa do arquivo indicado ou, sem argumento, da entrada padrão. O código de saída é 0 quando a entrada é aceita e 1 quando há erro, o que os testes automáticos usam.

Execução	Saída
Entrada correta	`entrada aceita`
Erro de sintaxe em `2 * (x + )`	`linha 1, coluna 10: encontrado ')', esperado um de {NUM, ID, '('}`
Caractere desconhecido em `3 + @`	`linha 1, coluna 5: erro lexico: caractere invalido '@'`

A mensagem de erro sintático diz onde o analisador parou e quais tokens ele aceitaria naquele ponto. Esse conjunto de tokens esperados vem dos conjuntos FIRST e FOLLOW que o gerador calculou.

Vendo a árvore de derivação

Se a gramática foi gerada com `--tree`, o analisador aceita a opção `--tree` e imprime a árvore de derivação. Cada nível de recuo é uma chamada de procedimento, e as folhas são os tokens consumidos. Para o arquivo `exemplo.txt` com `2 * x`:

```
$ ./build/expr_bin --tree exemplo.txt
expr
  term
    factor
      NUM '2'
    '*'
    factor
      ID 'x'
entrada aceita
```

A árvore é a melhor ferramenta para entender o comportamento do analisador. Se ela não tem o formato esperado, o problema está na gramática ou na tradução das regras.

7. Integrando o scanner do TP1 (modo Micro C)

No modo Micro C, o gerador não produz um analisador léxico. O analisador gerado chama a função `proximo_token()`, que encapsula o scanner entregue no TP1, sempre que precisa de um novo token. A gramática indica isso na seção de tokens, com `TOKENS EXTERN "tokens.h" ;`, e os terminais passam a ser os nomes da enumeração `TokenType`.

1. Copie para a pasta `microc/` o `microc.flex`, o `tokens.h` e os demais arquivos `.c` e `.h` de que o seu scanner depende, como a tabela de strings.
2. No `microc.flex`, use `%option prefix="microc_yy" noyywrap` e faça a regra `<<EOF>>` devolver `END_OF_FILE`.
3. Complete os `TODO(aluno)` de `microc/scanner.c`: o lexema deve vir da tabela de strings, e o token `UNDEF` deve trazer a mensagem de erro do TP1.
4. Rode `make microc`. O analisador fica em `build/microc/parser_microc`.

O gerador avisa sobre um conflito na gramática de Micro C. Esse aviso é esperado: é o *else* pendente, resolvido ligando o `else` ao `if` mais próximo, como em C.

```
microc.ebnf:19:57: aviso: conflito LL(1) em comando_if: FIRST e FOLLOW de [ ] se intersectam (resolvido de forma gulosa) em {'else'}
```

Como o scanner do TP1 só registra a linha de cada token, as mensagens de erro neste modo não têm coluna:

```
$ ./build/microc/parser_microc testes/programas/microc/rejeitado_sintaxe.mc
linha 3: encontrado ';', esperado um de {'(', ID, INTEGERCONST, CHARCONST, '!'}

$ ./build/microc/parser_microc testes/programas/microc/rejeitado_lexico.mc
linha 2: erro lexico: String nao terminada
```

Lendo o arquivo de tokens do TP1

Para depurar, o analisador também lê o arquivo de tokens gravado pelo TP1, com uma linha por token no formato do livro-texto:

```
Token: tipo = 33, lexema = 'int', linha = 1
```

Use `./build/microc/parser_microc --tokens tokens.txt`. A saída deve ser igual à da leitura direta do programa. Se as duas diferirem, o problema está no adaptador `scanner.c`, e não no analisador.

Cuidado com tokens devolvidos pelo próprio código ASCII, que o TP1 permite. Valores até o último item de `TokenType` são sempre lidos como `TokenType`, então `!` (código 33) deve ser devolvido como `NOT`.

8. Testes automatizados

O comando `make test` executa o gerador e os analisadores gerados sobre os casos da pasta `testes/` e compara cada saída com a esperada. No esqueleto, todos os testes falham. O número de testes que passam mede o seu progresso.

```
$ make test
== gramaticas
FALHA  gramatica conflito_ll1   (diferencas em
build/testes/conflito_ll1.gerador.diff)
FALHA  gramatica expr   (diferencas em build/testes/expr.gerador.diff)
...
pulado programas/microc (rode make microc com os arquivos do TP1)

0 de 11 testes passaram
```

Os programas de Micro C só são testados depois de `make microc`. Com eles, o total passa de 11 para 17 testes.

Lendo uma falha

Cada falha aponta um arquivo `.diff` em `build/testes/`. As linhas com `-` são o esperado, e as com `+` são o que o seu código produziu. A última linha de cada caso é o código de saída do programa.

```
-entrada aceita
-status 0
+linha 1, coluna 1: erro lexico: caractere invalido '2'
+status 1
```

Neste exemplo, o analisador ainda não reconhece `NUM`, porque o léxico gerado não sabe tratar expressões regulares.

Organização dos casos

Pasta	Conteúdo
`testes/gramaticas/`	Os arquivos `.ebnf` de teste
`testes/esperado/`	A saída esperada do gerador com `--sets`, um arquivo `.gerador` por gramática
`testes/programas/<gramatica>/`	Programas de entrada e, ao lado de cada um, a saída esperada do analisador (`.esperado`)

Para criar um caso, ponha o programa na pasta da gramática, por exemplo `testes/programas/expr/aceito_4.txt`. Execute o seu analisador, confira a saída à mão e salve-a como `aceito_4.esperado`, terminando com a linha `status 0` ou `status 1`. Para uma gramática nova, crie também a pasta de programas com o mesmo nome do arquivo `.ebnf`. O enunciado exige pelo menos 5 gramáticas e, para cada gramática válida, 3 programas aceitos e 3 rejeitados.

9. Roteiro de implementação

Implemente as partes na ordem abaixo. Cada etapa depende das anteriores, e cada uma tem um jeito simples de verificar o resultado antes de seguir.

Passo	Onde	Como saber que funcionou
1. Ler as expressões das regras	`parse_expr`, `parse_seq` e `parse_fator` em `ebnf_parser.c`	`--debug` imprime as regras de volta, iguais às do arquivo
2. Verificações semânticas	`verificacao.c`: inalcançáveis, improdutivos e recursão à esquerda	`recursao_esquerda.ebnf` é recusada com quatro erros
3. Anulável, FIRST e FOLLOW	`conjuntos.c`	A tabela de `--sets` coincide com o cálculo feito à mão
4. Condição LL(1)	`ll1.c`	`conflito_ll1.ebnf` é recusada, e o teste da gramática passa
5. Tradução das regras	`gen_expr` em `codegen.c`	O `.c` gerado tem `if` e `while` no lugar dos comentários `TODO`
6. Léxico próprio	`gen_casamento_regex` em `codegen.c`	Os 7 programas de `testes/programas/expr/` passam
7. Arquivo de tokens	`gen_token_de_arquivo` em `codegen.c`	`--tokens` dá a mesma saída que a leitura direta
8. Micro C	`microc_mapa.c` e `microc/scanner.c`	`make microc` compila e os 6 programas de Micro C passam

Conferindo os passos 1 e 3

Com a leitura pronta, `--debug` deve reproduzir a gramática. Os literais aparecem entre aspas simples:

```
$ ./gerador testes/gramaticas/expr.ebnf --debug
expr = term { ( '+' | '-' ) term } ;
term = factor { ( '*' | '/' ) factor } ;
factor = NUM | ID | '(' expr ')' ;
```

Com os conjuntos prontos, `--sets` deve mostrar:

```
NAO TERMINAL  ANULAVEL  FIRST / FOLLOW
expr          nao       FIRST  = {NUM, ID, '('}
                        FOLLOW = {$, ')'}
term          nao       FIRST  = {NUM, ID, '('}
                        FOLLOW = {$, '+', '-', ')'}
factor        nao       FIRST  = {NUM, ID, '('}
                        FOLLOW = {$, '+', '-', '*', '/', ')'}
```

Uma regra, uma função

O passo 1 é o mesmo método que o gerador vai produzir no passo 5: um analisador descendente recursivo, agora para a própria notação EBNF. A gramática da notação está no cabeçalho de `ebnf_parser.c`. Para a regra `seq = { fator } ;`, por exemplo:

```
função parse_seq():
    se o token atual não pode iniciar um fator: devolva um nó VAZIO
    leia o primeiro fator
    se o próximo token não inicia outro fator: devolva o fator sozinho
    crie um nó SEQUÊNCIA com o primeiro fator
    enquanto o token atual puder iniciar um fator:
        acrescente parse_fator() à sequência
    devolva a sequência
```

As funções que criam os nós (`expr_criar`, `expr_simbolo`, `expr_unario` e `expr_adicionar`) já estão prontas em `gramatica.c`.

10. Problemas comuns

Sintoma	Causa provável	O que fazer
`erro: nao foi possivel criar build/expr.h`	A pasta de saída não existe	Rode `make` ou `mkdir -p build`
O gerador fica parado e não termina	Um laço de ponto fixo que nunca para de "mudar", ou um laço de leitura que não avança o token	Interrompa com Ctrl+C e rode `gdb --args ./gerador ...`; o comando `bt` mostra onde estava
`Segmentation fault`	Ponteiro nulo, em geral uma regra ou um conjunto usado antes de ser criado	Rode com `valgrind ./gerador ...`, que mostra a linha do acesso inválido
Conjuntos vazios ou incompletos em `--sets`	O ponto fixo parou cedo, ou faltou calcular o FIRST de algum filho	Calcule à mão para uma gramática pequena e compare regra a regra
O analisador recusa uma entrada válida	Teste errado em uma alternativa, `[ ]` ou `{ }`	Rode com `--tree` e veja em que procedimento ele parou
`erro lexico: caractere invalido` em toda entrada	O léxico próprio ainda não reconhece as expressões regulares	Implemente `gen_casamento_regex` (passo 6)
Aviso de compilação no código gerado	O gerador emite C fora do padrão, como comentários `//` ou declarações no meio do bloco	Corrija o trecho de `codegen.c` que emite aquela linha
`make microc` pede o `tokens.h` ou o `microc.flex`	Os arquivos do TP1 não estão em `microc/`	Copie-os (seção 7)
O analisador de Micro C não termina no fim do arquivo	A regra `<<EOF>>` do flex não devolve `END_OF_FILE`	O flex devolve 0 no fim, que é `UNDEF` no `tokens.h`

Sintoma	Causa provável	O que fazer
`--tokens` e a leitura direta dão saídas diferentes	O adaptador `scanner.c` monta o token errado	Compare o tipo e o lexema de cada token nos dois modos

Para qualquer problema de memória, rode `valgrind --leak-check=full` antes de procurar o erro à mão. O trabalho exige zero vazamentos no gerador e no analisador gerado.

11. Checklist de entrega

Antes de entregar, confira cada item a partir de uma cópia limpa do repositório.

- ☐ `make clean && make` compila sem nenhum aviso.
- ☐ Nenhum `TODO(aluno)` sobrou: `grep -rn "TODO(aluno)" src microc` não devolve nada.
- ☐ `make test` passa em todos os testes, inclusive os de Micro C.
- ☐ Há pelo menos 5 gramáticas de teste: 3 válidas, uma delas a de Micro C, e 2 com erros (recursão à esquerda e conflito LL(1)).
- ☐ Cada gramática válida tem pelo menos 3 programas aceitos e 3 rejeitados, com saídas esperadas.
- ☐ O código gerado para cada gramática compila sem avisos com `gcc -ansi -pedantic -Wall -Wextra`.
- ☐ `valgrind --leak-check=full` não aponta vazamentos no gerador nem no analisador gerado.
- ☐ A pasta `microc/` contém o scanner do TP1 adaptado, com as correções feitas desde o TP1 descritas no `README.md`.
- ☐ O `README.md` traz as linhas `aluno:` de cada integrante e a linha `uso_ia:`.
- ☐ O `USO_IA.md` registra cada uso de ferramentas de IA, ou declara que nenhuma foi usada.
- ☐ O relatório de 4 a 8 páginas está no repositório.
- ☐ Todos do grupo sabem explicar e modificar qualquer parte do código para a apresentação.

Na apresentação, o professor fornecerá uma gramática nova, que deve ser processada ao vivo. Treine com gramáticas que você nunca testou.